

Node.js & Express.js

Complete Course Notes

From Basics to Production
By MUHAMMAD FARHAN

Covers: Node.js Core • NPM • Express.js • MongoDB • REST APIs • JWT • WebSockets • Deployment

1. Introduction to Node.js

What is Node.js?

Node.js is an open-source, cross-platform JavaScript runtime environment that executes JavaScript code outside the browser. It is built on Chrome's V8 JavaScript engine and uses an event-driven, non-blocking I/O model, making it ideal for scalable network applications.

History

Node.js was created by Ryan Dahl in 2009. He was frustrated with how Apache HTTP Server handled concurrent connections. Node.js introduced a new way to build fast, scalable servers using JavaScript on the server side.

Why Use Node.js?

- Same language (JavaScript) on both frontend and backend
- Non-blocking I/O — handles thousands of concurrent requests
- Huge ecosystem via npm (over 2 million packages)
- Fast performance powered by V8 engine
- Great for real-time apps, APIs, and microservices

Node.js vs Other Backend Technologies

Feature	Node.js	PHP	Python
Language	JavaScript	PHP	Python
Concurrency	Event-loop	Thread-based	Thread-based
Speed	Very Fast	Moderate	Fast
Use Case	APIs, RT apps	Web apps	AI, Web apps

2. Setting Up Environment

Install Node.js

Download from nodejs.org. Choose LTS (Long Term Support) version for stability.

- Windows: Download .msi installer and run it
- Mac: Use Homebrew: `brew install node`
- Linux (Ubuntu): `sudo apt install nodejs npm`

Verify Installation

```
# Check Node.js version
node -v
# Output: v20.x.x

# Check npm version
npm -v
# Output: 10.x.x
```

Running Your First Script

Create a file called `app.js`:

```
// app.js
console.log("Hello, Node.js!");

// Run it from terminal:
// node app.js
// Output: Hello, Node.js!
```

3. Basics of Node.js

Node REPL

REPL stands for Read-Eval-Print-Loop. It is an interactive shell to run JavaScript. Type 'node' in terminal to start.

```
$ node
> 2 + 3
5
> 'Hello'.toUpperCase()
'HELLO'
> .exit // to quit
```

Global Objects

```
// __dirname - current directory path
console.log(__dirname); // /home/user/project

// __filename - current file path
console.log(__filename); // /home/user/project/app.js

// process object
console.log(process.version); // Node.js version
console.log(process.env.NODE_ENV); // environment variable
console.log(process.argv); // command line arguments
```

4. Modules in Node.js

What are Modules?

Modules are reusable blocks of code. Node.js uses CommonJS module system. Each file is a separate module.

Built-in Modules

- fs — File System operations
- http — Create web servers
- path — Handle file paths
- os — Operating system info
- events — EventEmitter class

Creating Custom Modules

```
// math.js (custom module)
function add(a, b) {
  return a + b;
}
function subtract(a, b) {
  return a - b;
}
module.exports = { add, subtract };

// app.js (importing the module)
const math = require('./math');
console.log(math.add(5, 3)); // 8
console.log(math.subtract(5, 3)); // 2
```

ES Modules (import/export)

```
// math.mjs (ES Module)
export function add(a, b) { return a + b; }
export function multiply(a, b) { return a * b; }

// app.mjs
import { add, multiply } from './math.mjs';
console.log(add(2, 3)); // 5
console.log(multiply(2, 3)); // 6
```

5. NPM (Node Package Manager)

What is npm?

npm is the default package manager for Node.js. It has over 2 million open-source packages that you can install and use in your projects.

Essential npm Commands

<code>npm init</code>	# Create package.json (interactive)
<code>npm init -y</code>	# Create package.json with defaults
<code>npm install express</code>	# Install package (saves to dependencies)
<code>npm install --save-dev nodemon</code>	# Install dev dependency
<code>npm install -g nodemon</code>	# Install globally
<code>npm uninstall express</code>	# Remove a package
<code>npm update</code>	# Update all packages
<code>npm list</code>	# List installed packages
<code>npm run start</code>	# Run script from package.json

package.json Example

```
{
  "name": "my-app",
  "version": "1.0.0",
  "scripts": {
    "start": "node app.js",
    "dev": "nodemon app.js",
    "test": "jest"
  },
  "dependencies": {
    "express": "^4.18.2",
    "mongoose": "^8.0.0"
  },
  "devDependencies": {
    "nodemon": "^3.0.0"
  }
}
```

Tip: dependencies are needed in production. devDependencies are only for development (testing tools, hot-reload, etc.)

6. Asynchronous Programming

Callbacks

A callback is a function passed as an argument to another function, executed when an async operation completes.

```
const fs = require('fs');

// Callback style
fs.readFile('file.txt', 'utf8', (err, data) => {
  if (err) {
    console.error('Error:', err);
    return;
  }
  console.log(data);
});
```

Promises

Promises represent a future value. They can be in pending, fulfilled, or rejected state.

```
// Creating a Promise
function fetchData() {
  return new Promise((resolve, reject) => {
    setTimeout(() => {
      const data = { name: 'Alice', age: 25 };
      resolve(data); // success
      // reject(new Error('Failed')); // error
    }, 1000);
  });
}

// Using a Promise
fetchData()
  .then(data => console.log(data))
  .catch(err => console.error(err));
```

Async/Await

Async/Await is syntactic sugar over Promises. It makes async code look and behave like synchronous code.

```
async function getData() {
  try {
    const data = await fetchData();
    console.log(data);
  } catch (err) {
    console.error(err);
  }
}
```

```
getData();
```

Event Loop

The Event Loop is the core of Node.js. It allows Node.js to perform non-blocking operations by offloading tasks to the system kernel.

Call Stack → Node APIs → Callback Queue → Event Loop → Call Stack. This cycle enables Node.js to handle thousands of concurrent connections with a single thread.

7. File System (fs Module)

Reading Files

```
const fs = require('fs');

// Synchronous (blocking)
const data = fs.readFileSync('file.txt', 'utf8');
console.log(data);

// Asynchronous (non-blocking)
fs.readFile('file.txt', 'utf8', (err, data) => {
  if (err) throw err;
  console.log(data);
});

// Using Promises
const fsPromises = require('fs').promises;
async function readFile() {
  const data = await fsPromises.readFile('file.txt', 'utf8');
  console.log(data);
}
```

Writing Files

```
// Write file (overwrites if exists)
fs.writeFile('output.txt', 'Hello World!', (err) => {
  if (err) throw err;
  console.log('File written!');
});

// Append to file
fs.appendFile('output.txt', '\nNew line', (err) => {
  if (err) throw err;
  console.log('Data appended!');
});
```

Working with Directories

```
// Create directory
fs.mkdir('newFolder', (err) => {
  if (err) throw err;
  console.log('Directory created!');
});

// Read directory contents
fs.readdir('./', (err, files) => {
  if (err) throw err;
  console.log(files);
});
```

```
});  
  
// Delete file  
fs.unlink('file.txt', (err) => {  
  if (err) throw err;  
  console.log('File deleted!');  
});
```

Streams

```
// Read large file using streams (memory efficient)  
const readStream = fs.createReadStream('bigfile.txt', 'utf8');  
const writeStream = fs.createWriteStream('output.txt');  
  
readStream.on('data', (chunk) => {  
  console.log('Received chunk:', chunk.length);  
  writeStream.write(chunk);  
});  
  
readStream.on('end', () => console.log('Done reading'));  
  
// Pipe (simplest way)  
readStream.pipe(writeStream);
```

8. Creating a Web Server

HTTP Module Basics

```
const http = require('http');

// Create server
const server = http.createServer((req, res) => {
  // req = incoming request
  // res = outgoing response

  res.writeHead(200, { 'Content-Type': 'text/plain' });
  res.end('Hello from Node.js Server!');
});

// Listen on port 3000
server.listen(3000, () => {
  console.log('Server running at http://localhost:3000');
});
```

Basic Routing

```
const http = require('http');

const server = http.createServer((req, res) => {
  const url = req.url;
  const method = req.method;

  if (url === '/' && method === 'GET') {
    res.writeHead(200, { 'Content-Type': 'text/html' });
    res.end('<h1>Home Page</h1>');
  } else if (url === '/about' && method === 'GET') {
    res.writeHead(200, { 'Content-Type': 'text/html' });
    res.end('<h1>About Page</h1>');
  } else {
    res.writeHead(404, { 'Content-Type': 'text/plain' });
    res.end('404 Not Found');
  }
});

server.listen(3000);
```

9. Express.js Framework

What is Express.js?

Express.js is a minimal, fast, and unopinionated web framework for Node.js. It simplifies routing, middleware, and server creation.

```
npm install express
```

Basic Express App

```
const express = require('express');
const app = express();

// Middleware to parse JSON
app.use(express.json());

// GET route
app.get('/', (req, res) => {
  res.send('Hello from Express!');
});

// Start server
app.listen(3000, () => {
  console.log('Server running on port 3000');
});
```

Routing (GET, POST, PUT, DELETE)

```
// GET - Retrieve data
app.get('/users', (req, res) => {
  res.json([ { id: 1, name: 'Alice' }, { id: 2, name: 'Bob' } ]);
});

// GET with URL param
app.get('/users/:id', (req, res) => {
  const { id } = req.params;
  res.json({ id, name: 'Alice' });
});

// POST - Create data
app.post('/users', (req, res) => {
  const { name, email } = req.body;
  res.status(201).json({ message: 'User created', name, email });
});

// PUT - Update data
app.put('/users/:id', (req, res) => {
  const { id } = req.params;
  const { name } = req.body;
```

```

    res.json({ message: `User ${id} updated`, name });
  });

// DELETE - Remove data
app.delete('/users/:id', (req, res) => {
  const { id } = req.params;
  res.json({ message: `User ${id} deleted` });
});

```

Middleware

Middleware functions have access to req, res, and next(). They run in order and can modify request/response.

```

// Custom middleware - Logger
const logger = (req, res, next) => {
  console.log(`${req.method} ${req.url} - ${new Date().toISOString()}`);
  next(); // MUST call next() to continue
};

app.use(logger); // Apply globally

// Route-level middleware
const auth = (req, res, next) => {
  const token = req.headers['authorization'];
  if (!token) return res.status(401).json({ error: 'Unauthorized' });
  next();
};

app.get('/protected', auth, (req, res) => {
  res.json({ data: 'Secret data' });
});

// Built-in middleware
app.use(express.json()); // Parse JSON body
app.use(express.urlencoded({ extended: true })); // Parse form data
app.use(express.static('public')); // Serve static files

```

Express Router

```

// routes/users.js
const express = require('express');
const router = express.Router();

router.get('/', (req, res) => res.json({ msg: 'Get all users' }));
router.post('/', (req, res) => res.json({ msg: 'Create user' }));
router.get('/:id', (req, res) => res.json({ id: req.params.id }));

module.exports = router;

```

```
// app.js
const userRoutes = require('./routes/users');
app.use('/api/users', userRoutes);
```

10. Working with Databases

MongoDB with Mongoose

Mongoose is an ODM (Object Data Modeling) library for MongoDB and Node.js. It provides schema-based solution to model your data.

```
npm install mongoose
```

Connecting to MongoDB

```
const mongoose = require('mongoose');

mongoose.connect('mongodb://localhost:27017/mydb')
  .then(() => console.log('MongoDB Connected'))
  .catch(err => console.error('Connection error:', err));
```

Creating a Schema and Model

```
const mongoose = require('mongoose');

// Define Schema
const userSchema = new mongoose.Schema({
  name: { type: String, required: true },
  email: { type: String, required: true, unique: true },
  age: { type: Number, min: 0 },
  createdAt: { type: Date, default: Date.now }
});

// Create Model
const User = mongoose.model('User', userSchema);

module.exports = User;
```

CRUD Operations

```
const User = require('./models/User');

// CREATE
async function createUser() {
  const user = new User({ name: 'Alice', email: 'alice@example.com', age: 25 });
  await user.save();
  console.log('User created:', user);
}

// READ
async function getUsers() {
  const users = await User.find(); // All users
}
```

```
const user = await User.findById('id');    // By ID
const found = await User.findOne({ email: 'alice@example.com' });
}

// UPDATE
async function updateUser(id) {
  const updated = await User.findByIdAndUpdate(
    id,
    { age: 26 },
    { new: true } // return updated doc
  );
}

// DELETE
async function deleteUser(id) {
  await User.findByIdAndDelete(id);
  console.log('User deleted');
}
```


11. REST API Development

What is a REST API?

REST (Representational State Transfer) is an architectural style for designing networked applications. It uses HTTP methods and follows stateless communication.

HTTP Method	Action	Example
GET	Read/Retrieve	GET /api/users
POST	Create	POST /api/users
PUT	Update (full)	PUT /api/users/1
PATCH	Update (partial)	PATCH /api/users/1
DELETE	Delete	DELETE /api/users/1

Complete REST API with Express + MongoDB

```
const express = require('express');
const mongoose = require('mongoose');
const app = express();
app.use(express.json());

mongoose.connect('mongodb://localhost:27017/tododb');

const todoSchema = new mongoose.Schema({
  title: { type: String, required: true },
  completed: { type: Boolean, default: false }
});
const Todo = mongoose.model('Todo', todoSchema);

// GET all todos
app.get('/api/todos', async (req, res) => {
  const todos = await Todo.find();
  res.json(todos);
});

// POST create todo
app.post('/api/todos', async (req, res) => {
  const todo = new Todo(req.body);
  await todo.save();
  res.status(201).json(todo);
});

// PUT update todo
app.put('/api/todos/:id', async (req, res) => {
  const todo = await Todo.findByIdAndUpdate(req.params.id, req.body, { new: true });
  res.json(todo);
});
```

```
// DELETE todo
app.delete('/api/todos/:id', async (req, res) => {
  await Todo.findByIdAndDelete(req.params.id);
  res.json({ message: 'Deleted' });
});

app.listen(3000, () => console.log('API running on port 3000'));
```

12. Authentication & Security

JWT (JSON Web Tokens)

JWT is a compact, URL-safe token used for securely transmitting information. It has three parts: Header.Payload.Signature

```
npm install jsonwebtoken bcryptjs
```

Password Hashing with bcrypt

```
const bcrypt = require('bcryptjs');

// Hashing password
async function hashPassword(plain) {
  const salt = await bcrypt.genSalt(10);
  const hashed = await bcrypt.hash(plain, salt);
  return hashed;
}

// Comparing password
async function comparePassword(plain, hashed) {
  return await bcrypt.compare(plain, hashed);
}
```

Complete Auth System (Register + Login + Protected Route)

```
const jwt = require('jsonwebtoken');
const bcrypt = require('bcryptjs');
const SECRET = 'your_secret_key_here';

// User model (simplified)
const users = []; // In real app, use MongoDB

// Register
app.post('/auth/register', async (req, res) => {
  const { name, email, password } = req.body;
  const hashed = await bcrypt.hash(password, 10);
  const user = { id: Date.now(), name, email, password: hashed };
  users.push(user);
  res.status(201).json({ message: 'Registered!' });
});

// Login
app.post('/auth/login', async (req, res) => {
  const { email, password } = req.body;
  const user = users.find(u => u.email === email);
  if (!user) return res.status(404).json({ error: 'User not found' });

  const valid = await bcrypt.compare(password, user.password);
```

```
if (!valid) return res.status(401).json({ error: 'Wrong password' });

const token = jwt.sign({ id: user.id, email }, SECRET, { expiresIn: '1d' });
res.json({ token });
});

// Auth Middleware
function authenticate(req, res, next) {
  const token = req.headers.authorization?.split(' ')[1];
  if (!token) return res.status(401).json({ error: 'No token' });
  try {
    req.user = jwt.verify(token, SECRET);
    next();
  } catch {
    res.status(401).json({ error: 'Invalid token' });
  }
}

// Protected route
app.get('/profile', authenticate, (req, res) => {
  res.json({ message: 'Welcome!', user: req.user });
});
```

13. Advanced Concepts

EventEmitter

```
const EventEmitter = require('events');
const emitter = new EventEmitter();

// Register event listener
emitter.on('greet', (name) => {
  console.log(`Hello, ${name}!`);
});

// Emit event
emitter.emit('greet', 'Alice'); // Hello, Alice!

// One-time listener
emitter.once('startup', () => console.log('App started'));
emitter.emit('startup'); // logs
emitter.emit('startup'); // nothing (fires once)
```

Error Handling

```
// Global error handling middleware in Express
app.use((err, req, res, next) => {
  console.error(err.stack);
  res.status(500).json({ error: err.message || 'Internal Server Error' });
});

// Async error handling wrapper
const asyncHandler = (fn) => (req, res, next) => {
  Promise.resolve(fn(req, res, next)).catch(next);
};

app.get('/users', asyncHandler(async (req, res) => {
  const users = await User.find(); // if error, auto-caught
  res.json(users);
})));
```

Environment Variables

```
npm install dotenv

// .env file
PORT=3000
MONGODB_URI=mongodb://localhost:27017/mydb
JWT_SECRET=supersecretkey
NODE_ENV=development

// app.js
```

```
require('dotenv').config();
```

```
const PORT = process.env.PORT || 3000;
```

```
const MONGODB_URI = process.env.MONGODB_URI;
```

```
const SECRET = process.env.JWT_SECRET;
```

14. Real-Time Applications with Socket.io

Setup

```
npm install socket.io
```

Basic Chat Application

```
// server.js
const express = require('express');
const http = require('http');
const { Server } = require('socket.io');

const app = express();
const server = http.createServer(app);
const io = new Server(server);

app.get('/', (req, res) => res.sendFile(__dirname + '/index.html'));

io.on('connection', (socket) => {
  console.log('User connected:', socket.id);

  // Receive message from client
  socket.on('chat message', (msg) => {
    console.log('Message:', msg);
    io.emit('chat message', msg); // Broadcast to all
  });

  socket.on('disconnect', () => {
    console.log('User disconnected:', socket.id);
  });
});

server.listen(3000, () => console.log('Server on port 3000'));

// index.html (client side)
// <script src='/socket.io/socket.io.js'></script>
// const socket = io();
// socket.emit('chat message', 'Hello everyone!');
// socket.on('chat message', msg => console.log(msg));
```

15. Deployment

Preparing for Production

```
// Use environment variable for port
const PORT = process.env.PORT || 3000;

// Handle uncaught exceptions
process.on('uncaughtException', (err) => {
  console.error('Uncaught Exception:', err);
  process.exit(1);
});

// Graceful shutdown
process.on('SIGTERM', () => {
  server.close(() => {
    mongoose.connection.close();
    process.exit(0);
  });
});
```

PM2 — Process Manager

```
npm install -g pm2

pm2 start app.js          # Start app
pm2 start app.js --name myapp # Start with name
pm2 list                  # List running apps
pm2 stop myapp            # Stop app
pm2 restart myapp         # Restart app
pm2 logs myapp            # View logs
pm2 startup               # Auto-start on server reboot
pm2 save                  # Save process list
```

Deployment Checklist

- Set NODE_ENV=production
- Use .env file for secrets (never commit it to Git)
- Add .gitignore with node_modules and .env
- Use PM2 for process management
- Set up Nginx as reverse proxy
- Enable HTTPS with SSL certificate
- Monitor with PM2 or cloud tools

16. Testing & Debugging

Debugging with Node.js

```
// Built-in debugger
node --inspect app.js
// Then open chrome://inspect in Chrome

// console debugging
console.log('Value:', someVar);
console.error('Error occurred');
console.time('operation');
// ... code
console.timeEnd('operation'); // logs elapsed time
```

Unit Testing with Jest

```
npm install --save-dev jest

// math.js
function add(a, b) { return a + b; }
module.exports = { add };

// math.test.js
const { add } = require('./math');

test('adds 2 + 3 to equal 5', () => {
  expect(add(2, 3)).toBe(5);
});

test('adds negative numbers', () => {
  expect(add(-1, -2)).toBe(-3);
});

// Run tests
// npx jest
```

API Testing with Supertest

```
npm install --save-dev supertest

const request = require('supertest');
const app = require('./app');

test('GET /api/users returns 200', async () => {
  const res = await request(app).get('/api/users');
  expect(res.statusCode).toBe(200);
  expect(Array.isArray(res.body)).toBe(true);
});
```

```
});  
  
test('POST /api/users creates user', async () => {  
  const res = await request(app)  
    .post('/api/users')  
    .send({ name: 'Alice', email: 'alice@test.com' });  
  expect(res.statusCode).toBe(201);  
});
```

17. MVC Pattern & Project Structure

Recommended Folder Structure

my-app/	
├─ controllers/	# Route handlers / business logic
└─ userController.js	
└─ authController.js	
├─ models/	# Mongoose schemas
└─ User.js	
├─ routes/	# Express routers
└─ userRoutes.js	
└─ authRoutes.js	
├─ middleware/	# Custom middleware
└─ auth.js	
└─ errorHandler.js	
├─ config/	# DB connection, env config
└─ db.js	
├─ .env	
├─ .gitignore	
├─ app.js	# Express app setup
├─ server.js	# Entry point
└─ package.json	

MVC Example

// models/User.js
const mongoose = require('mongoose');
const userSchema = new mongoose.Schema({
name: String, email: String, password: String
});
module.exports = mongoose.model('User', userSchema);
// controllers/userController.js
const User = require('../models/User');
exports.getAllUsers = async (req, res) => {
const users = await User.find().select('-password');
res.json(users);
};
exports.getUserById = async (req, res) => {
const user = await User.findById(req.params.id);
if (!user) return res.status(404).json({ error: 'Not found' });
res.json(user);
};
// routes/userRoutes.js
const express = require('express');
const router = express.Router();
const { getAllUsers, getUserById } = require('../controllers/userController');

```
router.get('/', getAllUsers);  
router.get('/:id', getUserById);
```

```
module.exports = router;
```

```
// app.js
```

```
const userRoutes = require('./routes/userRoutes');  
app.use('/api/users', userRoutes);
```

18. Best Practices

Code Quality

- Use async/await instead of callbacks
- Always handle errors with try/catch or error middleware
- Validate inputs before processing (use express-validator or Joi)
- Never store secrets in code — use .env files
- Use consistent naming: camelCase for variables, PascalCase for classes

Security

- Use helmet to set security HTTP headers
- Rate limit APIs to prevent abuse (express-rate-limit)
- Sanitize inputs to prevent NoSQL injection
- Use HTTPS in production
- Hash passwords — never store plaintext

Performance

- Use caching (Redis) for frequent database queries
- Use connection pooling for database
- Compress responses with compression middleware
- Use streams for large file transfers
- Avoid synchronous operations in route handlers

Quick Reference — Security Packages

```
npm install helmet          # Security headers
npm install cors            # Cross-Origin Resource Sharing
npm install express-rate-limit # Rate limiting
npm install express-validator # Input validation
npm install compression     # Response compression

// app.js
const helmet = require('helmet');
const cors = require('cors');
const rateLimit = require('express-rate-limit');

app.use(helmet());
app.use(cors());
app.use(rateLimit({ windowMs: 15 * 60 * 1000, max: 100 })); // 100 req/15min
```